

Sound + Animation

making the brick-breaker feel alive

pygame.mixer · SFX & music · frame-based sprite animation · event triggers

Day 10 gave us a silent brick-breaker

WHAT WORKS NOW

Brick-breaker prototype

- Paddle moves with the mouse
- Ball bounces off paddle, walls, bricks
- Bricks vanish on hit (Group.remove)
- Rect.colliderect + spritecollide wired in
- Ball off-screen → game over

WHAT'S MISSING

It's silent and lifeless

- No collision feedback — feels dead
- Paddle is a static rectangle on every hit
- No background music or atmosphere
- Player can't tell whether they hit or grazed
- Today: fix all four

Initialise the audio system before anything plays

pygame's audio engine needs to start up before any sound loads. `pre_init` lets you set the buffer size first, which controls how fast SFX trigger when the ball hits the paddle.

```
import pygame

# pre_init sets audio buffer BEFORE pygame.init()
# small buffer = low latency, more CPU work
pygame.mixer.pre_init(
    frequency=44100, # 44.1 kHz, CD quality
    size=-16,       # signed 16-bit samples
    channels=2,      # stereo
    buffer=512       # samples per buffer
)
pygame.init()       # initialises mixer too
```

ANATOMY

- **frequency** samples/sec. 44100 is the standard.
- **buffer** the key knob. 512 $\approx$ 12 ms lag, sweet spot for games.
- **size = -16** 16-bit signed PCM, what most .wav files use.
- **pygame.init()** calls `mixer.init()` for you using `pre_init` values.

Sound objects: load once, play many times

A Sound object loads a whole audio file into RAM once. After that, `.play()` fires it instantly without re-reading the file. This is the right pattern for short repeated SFX like a paddle hit.

```
# load ONCE at startup, never inside the loop
hit = pygame.mixer.Sound("sfx/hit.wav")
break_ = pygame.mixer.Sound("sfx/break.wav")
lose = pygame.mixer.Sound("sfx/lose.wav")

hit.set_volume(0.6) # 0.0 to 1.0

# inside the game loop, on collision:
if ball.rect.colliderect(paddle.rect):
    hit.play() # fires and forgets

if pygame.sprite.spritecollide(ball, bricks, True):
    break_.play()
```

KEY IDEAS

- **Load at startup** `Sound()` inside the loop re-reads the file — kills framerate.
- **`.play()` is fire-and-forget** returns a Channel you can ignore for one-shots.
- **Concurrent SFX** many Sound objects can play at the same time.
- **Volume per Sound** `set_volume(0.6)` keeps loud SFX from blowing ears out.

mixer.music streams instead of loading

pygame.mixer.music is a separate module for the background track. Unlike Sound objects, it streams from disk while playing so a 3-minute song doesn't sit in RAM. Use it for music, never for SFX.

```
# music is a MODULE, not a class — no constructor
pygame.mixer.music.load("music/bg_loop.ogg")
pygame.mixer.music.set_volume(0.3)
pygame.mixer.music.play(loops=-1) # -1 = forever

# pause / resume / stop work as you'd expect
pygame.mixer.music.pause()
pygame.mixer.music.unpause()
pygame.mixer.music.stop()

# query state
if not pygame.mixer.music.get_busy():
    pygame.mixer.music.play()
```

WHEN TO USE WHAT

- **Sound** short clips, < 5 sec, fired many times.
- **music** one long track, streamed, looped.
- **Only one music track at a time** `load()` overwrites the current track.
- **.ogg over .mp3** better cross-platform support, no licensing.

An animation is a list of images shown in sequence

Frame animation = a list of images + a timer that advances the index every N milliseconds. Splitting the animation clock from the render clock means the animation looks identical at 30, 60, or 144 FPS.

```
# load all frames of the squish strip into a list
frames = [
    pygame.image.load(f"paddle/squish_{i}.png").convert_alpha()
    for i in range(5)
]

frame_index = 0    # which frame to draw now
frame_timer = 0    # ms since last frame switch
FRAME_MS = 80     # 80 ms per frame → 12.5 fps

# every game loop tick:
dt = clock.tick(60)    # ms since last frame
frame_timer += dt
if frame_timer >= FRAME_MS:
    frame_timer = 0
    frame_index = (frame_index + 1) % len(frames)

screen.blit(frames[frame_index], pos)
```

THE PATTERN

- **clock.tick(60)** returns ms since last frame AND caps at 60 fps.
- **Modulo loops** $(idx + 1) \% len \rightarrow$ wraps back to frame 0 forever.
- **80 ms / frame** smooth at any render rate, animation timing is independent.
- **convert_alpha()** pre-converts the image for fast blitting with transparency.

Wrap the frame logic inside a Sprite subclass

Now move the animation state INTO a Sprite. Each animated object owns its own frame counter and timer. The Group's `draw()` reads `self.image` and `self.rect` — mutating `self.image` inside `update()` is what makes the animation visible.

```
class AnimatedPaddle(pygame.sprite.Sprite):
    def __init__(self, frames, pos):
        super().__init__()
        self.frames = frames
        self.idx = 0
        self.timer = 0
        self.frame_ms = 80
        self.image = frames[0]
        self.rect = self.image.get_rect(center=pos)

    def update(self, dt):
        self.timer += dt
        if self.timer >= self.frame_ms:
            self.timer = 0
            self.idx = (self.idx + 1) % len(self.frames)
            self.image = self.frames[self.idx]
```

WHY IT WORKS

- **image + rect** the two attributes `Group.draw()` reads. That's it.
- **update(dt)** `Group.update(dt)` calls this on every sprite, passing `dt` through.
- **State stays internal** `frame`, `timer`, and `frame_ms` are per-paddle.
- **No screen access** the sprite never blits. The renderer reads `.image` and draws.

Play once on hit, then return to idle

A squish shouldn't loop forever. Add a 'playing' flag: only advance frames while it's True. When you reach the last frame, snap back to frame 0 and shut off. Call squish() from outside to trigger.

```
class AnimatedPaddle(pygame.sprite.Sprite):
    def __init__(self, frames, pos):
        super().__init__()
        self.frames, self.idx, self.timer = frames, 0, 0
        self.playing = False          # start at idle frame
        self.image = frames[0]
        self.rect = self.image.get_rect(center=pos)

    def squish(self):                  # called on collision
        self.idx, self.timer = 0, 0
        self.playing = True

    def update(self, dt):
        if not self.playing: return
        self.timer += dt
        if self.timer >= 80:
            self.timer, self.idx = 0, self.idx + 1
            if self.idx >= len(self.frames):
                self.idx, self.playing = 0, False
            self.image = self.frames[self.idx]
```

TRIGGER FROM OUTSIDE

```
# on paddle hit:
if ball.rect.colliderect(
    paddle.rect):
    paddle.squish()
    hit.play()
```

One call to squish() resets the timer and flips the flag. update() drains the animation and turns itself off.

Bring the brick-breaker to life

Take yesterday's silent prototype and add three sounds + a paddle squish animation. Don't refactor the collision logic — just hook into the existing collision branches.

SOUND ASSETS

Three .wav files in sfx/

- hit.wav — short blip (paddle bounce)
- break.wav — snap (brick destroy)
- lose.wav — descending tone (ball missed)
- Free from freesound.org / zapsplat

SPRITE FRAMES

5-frame paddle squish strip

- Frame 0 — idle (default shape)
- Frames 1-2 — compress vertically
- Frames 3-4 — recoil / overshoot
- Same width per frame, transparent bg

WIRE-UP

Hook into collision branches

- Startup: pre_init, init, load Sounds
- Ball-paddle: hit.play() + squish()
- Ball-brick: break_.play() + kill()
- Off-screen: lose.play() + game-over

Four things that will bite you

⚠ **Never load sounds inside the game loop**

Sound() re-reads the file from disk every call. Three SFX × 60 fps = 180 disk reads per second. Your framerate collapses. Load all Sound objects ONCE at startup and keep them in module-level variables.

⚠ **buffer too large = noticeable lag**

buffer=4096 in pre_init gives ~93 ms latency. The ball will visibly hit the paddle before you hear it. Use 512 (≈12 ms). Go lower (256) only if you hear no glitches and need tighter sync.

💡 **set_num_channels for many concurrent SFX**

Default is 8 channels. If you spawn 20 brick-break sounds in one frame, the older ones get cut off. pygame.mixer.set_num_channels(16) raises the cap. Don't go crazy — each channel uses memory.

💡 **Prefer .ogg and .wav over .mp3**

.mp3 has historical patent/codec issues across platforms and pygame versions. .wav is uncompressed (fine for short SFX). .ogg is compressed and cross-platform (best for music). Skip .mp3 unless you have to use it.

Today plugs into the rest of week 2

DAY 12 — FRI

Scene state machine

Wrap today's loop in a Scene base class. Menu / play / game-over each become a Scene with `handle_event`, `update`, `draw`. The `lose.play()` trigger becomes a scene transition.

DAY 13 — SAT

Pong proper

Two paddles, ball-paddle deflection angle, scoring, simple AI. Sound + animation patterns from today drop straight in. The menu wraps it via Day 12's scenes.

DAY 14 — SUN

Stretch pick

Option A: Asteroids-lite with vector rotation and screen wrap. Option B: brick-breaker polish + JSON level loader. Decide before EOD Wednesday.

By Saturday, you'll have a real Pong with menus, sound, animation, and scoring — built entirely on this week's patterns.